

Advanced Fuzzing and Crash Analysis

© Richard Johnson 2019

rjohnson@fuzzing.io

Day 1

Agenda

- Analysis of generational and mutational fuzzing
- Fuzzing file and network parsers with coverage guided fuzzing
- Best practices for high performance fuzzing

Introduction



- Richard Johnson
Director, Oracle Cloud
- @richinseattle
rjohnson@fuzzing.io

whoami

- Oracle Cloud :: Security Research
 - Deep Dives on Oracle Cloud
 - Every major component including hypervisors, kernels, userland, web services
 - Third party vulnerability research
 - Major internet and cloud libraries
 - Third party dependencies
 - Security tool development
 - Fuzzers, Crash Triage, Static Analysis
 - Mitigation development

Class Member Introductions

Vulnerability Research Strategies

Vulnerability Research Strategies

- Vulnerability research has many approaches with various tradeoffs
- Typically the primary tradeoffs include cost in time, scalability, reproducibility, precision, and targeting

Vulnerability Research Strategies

- Manual Data Flow Analysis
 - Can be performed on any form of code
 - Produces an undefined number of bugs
 - Manual efforts are not repeatable or scalable
 - Very expensive and limited source of engineers
- Static Data Flow Analysis
 - Can target classes of bugs
 - Automated and repeatable
 - High false positive rate
 - Lacking effective algorithms
- Dynamic Data Flow Analysis
 - Can target classes of bugs
 - Automated and repeatable
 - Solves some problems with static analysis
 - Lacking effective algorithms*

```
int main ( int argc, char **argv )
{
    FOO_STRUCT foo;
    ...
    foo.val = strdup(argv[1]);
    foo.sz = strlen(foo.val);
    ...
    vuln(&foo);
}

void vuln ( struct *foo )
{
    char buf[STATIC_SIZE];
    ...
    strncpy(buf, foo->val, foo->sz);
}
```

Vulnerability Research Strategies

- For some applications, vulnerability research requires finding all the bugs for others one exploitable one will do
- The impact of a bug may have varying degrees of criticality depending on the application of the technology
- If the software is a critical component with human safety involved, such as on a flight navigation computer it may be serious, but if it just crashes your browser, it's less serious

Vulnerability Research Strategies

- There are many nice strategies we will not cover
 - Manual source or binary code review
 - Using graph query languages to find patterns in source code
 - Writing proofs for verifiable implementations
 - Annotating source code for property checking

Vulnerability Research Strategies

- This week we will focus on strategies that can be mostly automated and analyze these approaches for ways to optimize our tactics in executing a strategy
- We will cover some approaches for detecting vulnerabilities in source code, however due to time constraints the primary focus will be strategies that can be leveraged against binary targets

Vulnerability Research Strategies

- We will be looking for any practical bugs that would be classified as a vulnerability, our primary tool of choice will be fuzzing
- We will learn how to maximize the effectiveness of off-the-shelf tools through input selection, modifying our targets, or extracting metadata about the target that can help guide fuzzing
- We will learn how to extend grammars and how to hook deeper into programs to target specific areas of code

Fuzzing Introduction

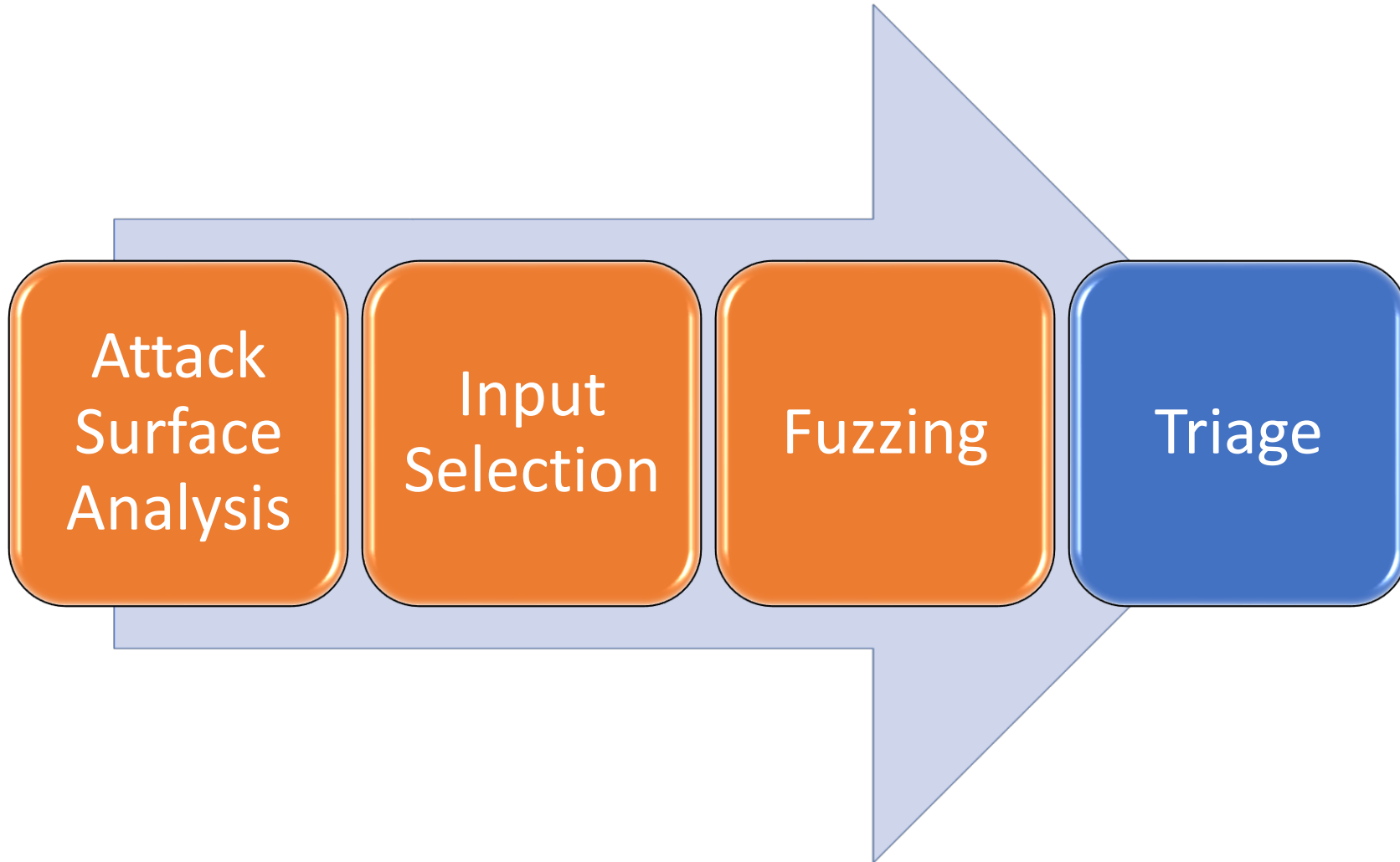
Fuzzing Introduction

- Fuzzing is a method of software testing
- A high volume of *exceptional data* is sent to various interfaces of a target to locate faulty program logic
- Simple in concept, complex in practice
 - Hundreds of fuzzers have been written
- Fuzzing has held up in practical testing
 - Many thousands of bugs have been identified

Fuzzing Introduction

- Identifying flaws in software is critical to the reliability and security of our information systems
- Security critical bugs are very expensive to fix in deployed products
- Fuzzers produce repeatable results useful for regression testing
- Fuzz testing is part of the SDL best practices

The Fuzzing Process



Keys to Fuzzing Smartly

- Input selection
 - Most important factor in timely bug discovery
 - Time to achieve data coverage is relative to number of bytes
 - Number of states in target determined by the selected inputs
- Automation Opportunities
 - Fuzzing
 - Crash deduplication
 - Root cause analysis

Input Selection

- Attack Surface Analysis
 - Determine which areas of the code are reachable from external inputs
- Input set code coverage
 - Determine what areas of code are exercised by different samples
- We want the minimal number of samples that exercise the code we are targeting

Feedback Driven Fuzzing

The Miller Theorem

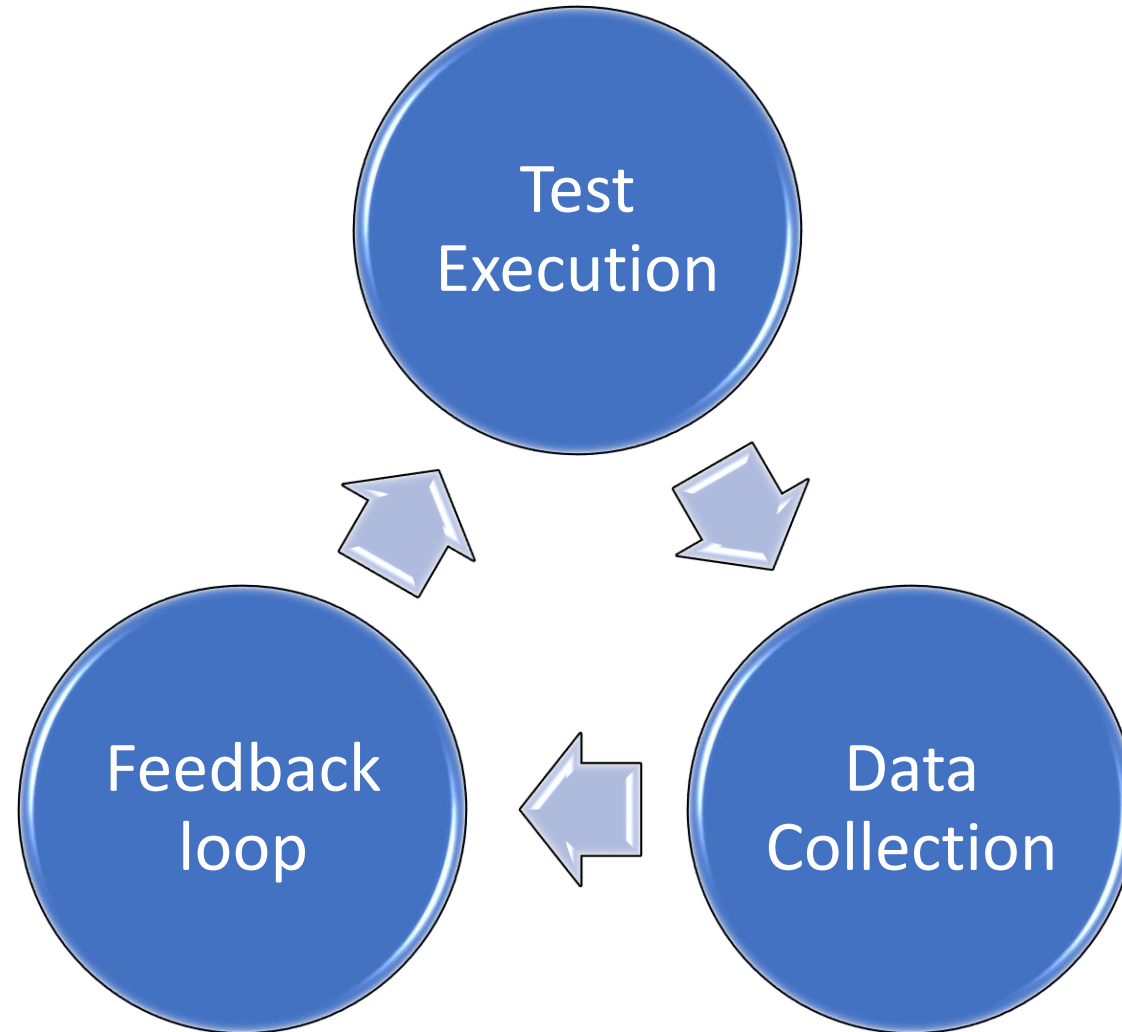
C = code path coverage

T = Time spent Fuzzing

B = Bugs Discovered

$$\partial C \cdot \partial T = \partial B$$

Feedback Driven Fuzzing



Classic Fuzzing Technology

Initial Public Offering

- Barton Miller, et al “An Empirical Study of the Reliability of UNIX Utilities”, 1990
- Targeted command line argument parsing on 90 console utilities in 7 UNIX varieties
- Introduced “fuzz”, the first dumb fuzzer
- Results: 25% – 33% of the utilities tested crashed, depending on the version of UNIX
- Fuzzed with unstructured, random data

“Our approach is not a substitute for a formal verification or testing procedures, but rather an inexpensive mechanism to identify bugs and increase overall system reliability.”

Initial Public Offering

- Miller tried again in 1995 with improvements
 - X Windows clients
 - Network ports
 - Memory exhaustion simulation
- Crashed as many as 40% of the console utilities and 25% X windows clients
- None of the network facing code faulted

“Our 1995 study surprised us ... the continued prevalence of bugs in the basic UNIX utilities seems a bit disturbing. The simplicity of performing random testing and its demonstrated effectiveness would seem to be irresistible to corporate testing groups.”

Fuzzy Input

- Miller, inspired by the storm, used random input data
- *Mutation* based input performs transformations on existing protocol data
- Static lists of values are used to target common implementation defects and known classes of bugs

Smarter Fuzzing

- Fuzzing interfaces with unstructured inputs will yield limited results
- Structured inputs allow for more effective traversal of program states
- This is where the art of fuzzing begins

Origins of Smart Fuzzing

- SPIKE, Dave Aitel, 2002
 - C language API for data generation and rapid network client development
 - Structured data dynamically defined as blocks
 - Relation model for size fields
- Peach Fuzzer Framework, Michael Eddington, 2004
 - Object oriented python API
 - Improved block based analysis with an abstracted fuzzing model

Origins of Smart Fuzzing

- **PROTOS, 2002**

Functional fuzzing using behavior models

Master Specification

- BNF notation utilized to describe *interaction models* and *syntax models*

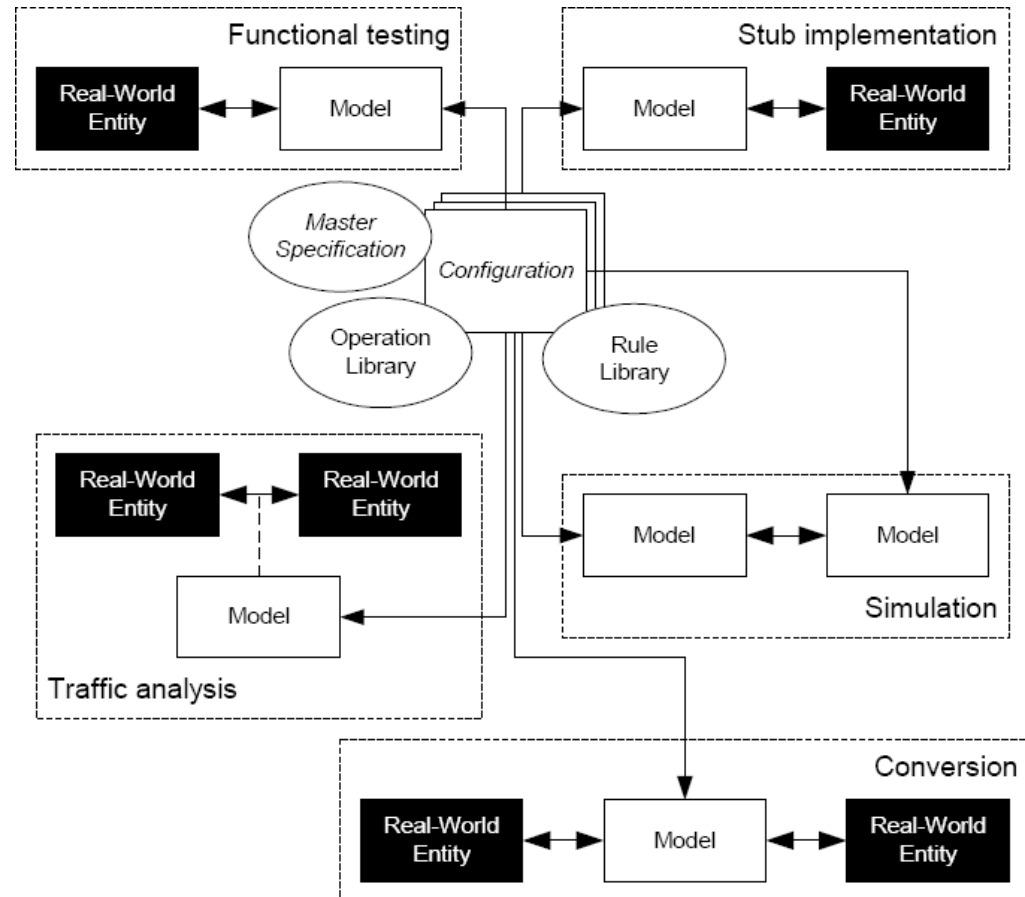
Configuration

- Performs *operations* on the master specification to derive a Mini-Simulation model

Communication Rules

- Connect the model to execution environment

PROTOS Mini-Simulation Concept



“A Functional Method for Assessing Protocol Implementation Security”, Rauli Kaksonen

Origins of Smart Fuzzing

- Entity Modeling
 - Describes internal behavior of an entity
 - Standards
 - Specification and Description Language (SDL)
 - Unified Modeling Language (UML)
- Interaction Modeling
 - Describes behavior between two entities
 - Standards
 - Unified Modeling Language (UML)
 - Tree and Tabular Combined Notation (TTCN)
 - Message Sequence Chart (MSC)
- Syntax Modeling
 - Describes the structure of data exchanged by entities
 - Standards
 - Abstract Syntax Notation One (ASN.1)
 - Extensible Markup Language (XML)

Origins of Smart Fuzzing

- Behavior Modeling

Backus-Naur Form (BNF)

- Flexible context-free grammar extension to regular expressions
- Lacking standard notation

Simulation Grammar

- Attribute grammar using modified BNF notation
- Tree-based *Data Productions*
- *Tags* represent callbacks such as *input triggers*

PROTOS Mini-Simulation Behavior Grammar (TFTP)

```
<transfer> = <read-transfer> | <write-transfer>
<read-transfer> = !up<RRQ> <reads>
<write-transfer> = !up<WRQ> <writes>
<reads> = {!down<BLOCK> !up<ACK>} !down<LAST-BLOCK> !up<ACK>
<writes> = !down<ACK> {!up<BLOCK> !down<ACK>} !up<LAST-BLOCK> !down<ACK>
```

Origins of Smart Fuzzing

- Behavior Modeling

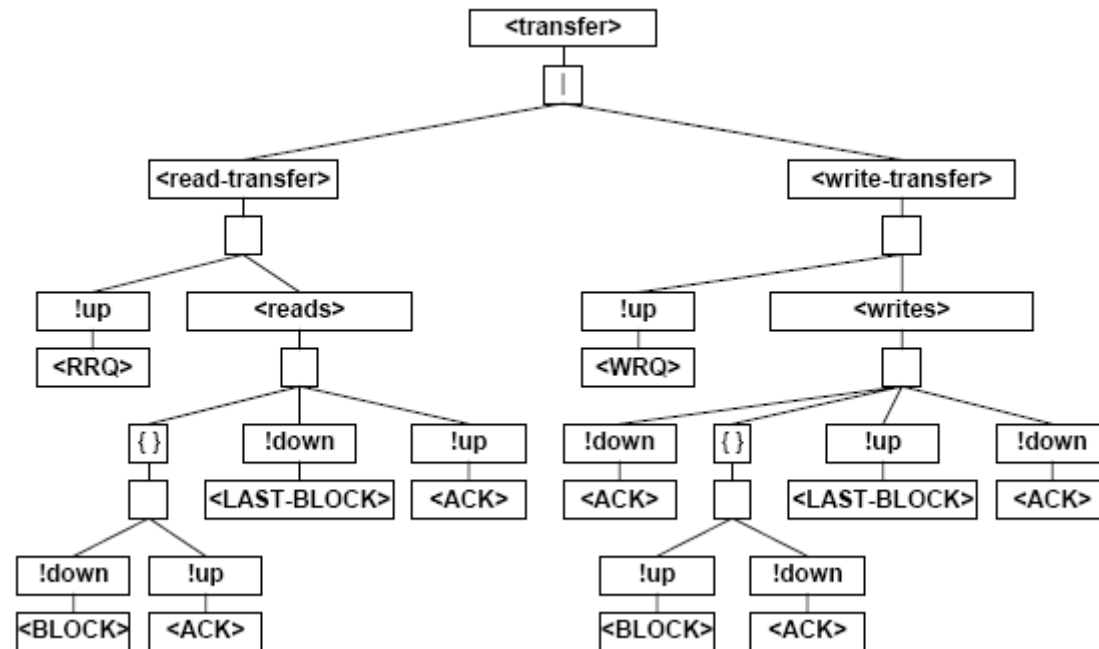
Backus-Naur Form (BNF)

- Flexible context-free grammar extension to regular expressions
- Lacking standard notation

Simulation Grammar

- Attribute grammar using modified BNF notation
- Tree-based *Data Productions*
- *Tags* represent callbacks such as *input triggers*

PROTOS Mini-Simulation Behavior Tree (TFTP)



Origins of Smart Fuzzing

- Syntax Modeling

Syntax Grammar

- Also uses modified BNF
- Tree-based *Type Productions*

Evaluation

- Transforms input grammar to output grammar
- Engine traverses input tree, executing *rules* on subtrees
- *Semantic Rules* evaluate data
- *Communication Rules* implement I/O

PROTOS Mini-Simulation Syntax Grammar (TFTP)

```
# Request PDUs
<RRQ> ::= (0x00 0x01) <FILE-NAME> <MODE>
<WRQ> ::= (0x00 0x02) <FILE-NAME> <MODE>

# Subsequent PDUs
<BLOCK> ::= (0x00 0x03) <BLOCK-NUMBER> 512 x <OCTET>
<LAST-BLOCK> ::= (0x00 0x03) <BLOCK-NUMBER> 0..511 { <OCTET> }
<ACK> ::= (0x00 0x04) <BLOCK-NUMBER>
<ERROR> ::= (0x00 0x05) <ERROR-CODE> <ERROR-MESSAGE>

# Miscellaneous productions
<MODE> ::= "octet" 0x00 | "netascii" 0x00
<FILE-NAME> ::= { <CHARACTER> } 0x00
<BLOCK-NUMBER> ::= <OCTET> <OCTET>
<ERROR-CODE> ::= <OCTET> <OCTET>
<ERROR-MESSAGE> ::= { <CHARACTER> } 0x00
<CHARACTER> ::= 0x01 - 0x7f
<OCTET> ::= 0x00 - 0xff
```

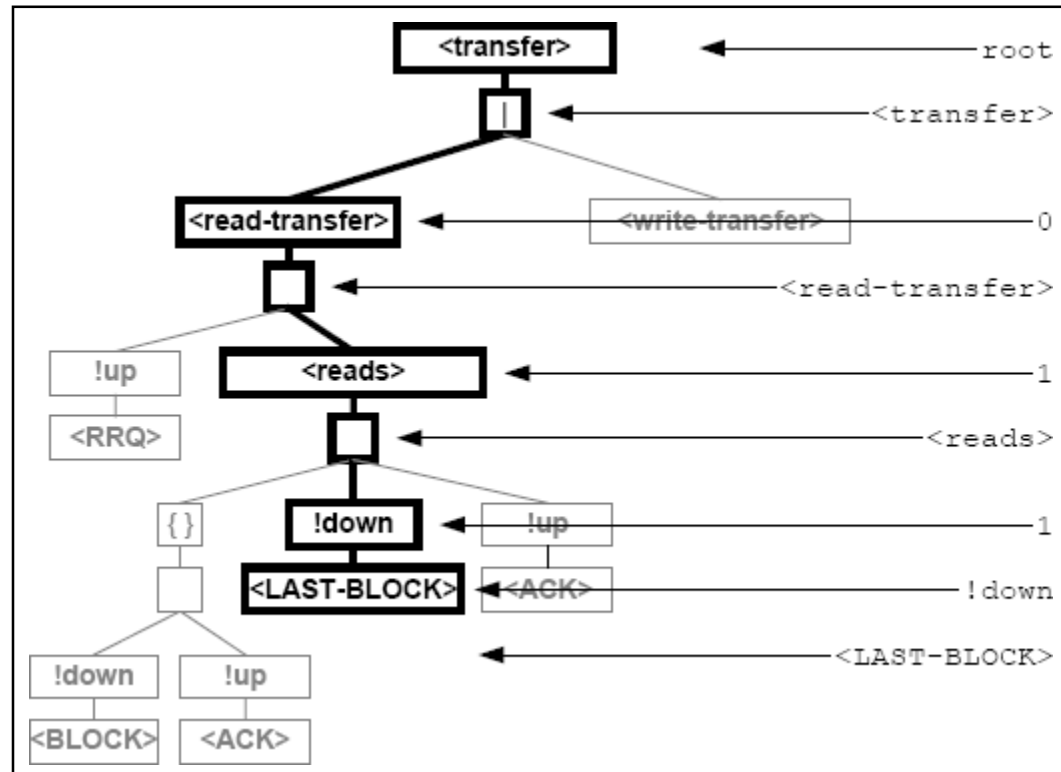
Origins of Smart Fuzzing

- State Traversal

Path Finding

- Paths are used to access elements of the grammar
- Masks can be used as an optimized path representation

PROTOS Mini-Simulation Path Representation



Test case generation

- Generational fuzzing takes a data model or grammar as input and creates permutations of the defined types while potentially introducing randomness or fuzzy values into the generated output
- Grammars can be specified in various formats
 - BNF – Backus-Naur Form is a Context Free Grammar popular in RFCs
 - XML – Peach and others use this
 - Native C / Python data structures – Spike, Sulley, Boo, etc

Test case generation

- Generational fuzzing was also referred to as smart fuzzing due to the integral knowledge of the expected data structure by the fuzzer
- Currently generational fuzzing is most popular for fuzzing interpreters and scripting engines or abstract type trees like DOMs
- In a later segment we will discuss fuzzing browsers with grammars and data models

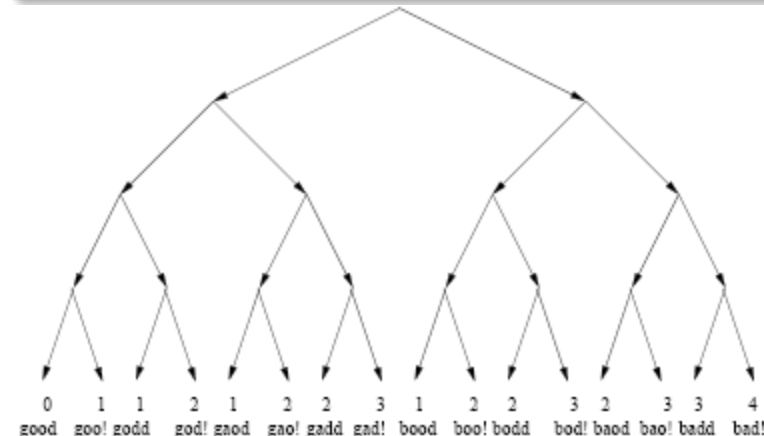
Automated White Box Fuzzing

- Scalable, Automated, Graph Execution (SAGE)

“Automated Whitebox Fuzz Testing”, Godefroid, Levin, Molnar 2006

- Runtime state of a recorded session is stored for analysis
- Symbolic execution gathers input constraints from conditional statements
- Solution given by known-good input data is negated and solved again
- *Generational vs Depth-First Search (DFS) algorithms*

```
void top(char input[4])
{
    int cnt = 0;
    if (input[0] == 'b') cnt++;
    if (input[1] == 'a') cnt++;
    if (input[2] == 'd') cnt++;
    if (input[3] == '!') cnt++;
    if (cnt >= 3) abort();
}
```



Classic fuzzing engines today

Radamsa

- Dumb mutation can still be useful in some cases, radamsa is considered by many to be the most useful off-the-shelf mutator
 - Was used as part of Trail of Bits Cyber Grand Challenge strategy

```
$ radamsa -o out/test_%n.html -n inf -r input/html/*.html -M -
```

```
-o output
```

```
%n represents test case number
```

```
-n number of outputs (-1 or inf generates infinite)
```

```
-M output metadata to file
```

Radamsa

```
$ radamsa -l
```

```
Mutations (-m)
```

```
ab: enhance silly issues in ASCII string data handling
```

```
bd: drop a byte
```

```
bf: flip one bit
```

```
bi: insert a random byte
```

```
br: repeat a byte
```

```
bp: permute some bytes
```

```
bei: increment a byte by one
```

```
bed: decrement a byte by one
```

```
ber: swap a byte with a random one
```

Radamsa

sr: repeat a sequence of bytes
sd: delete a sequence of bytes
ld: delete a line
lds: delete many lines
lr2: duplicate a line
li: copy a line closeby
lr: repeat a line
ls: swap two lines
lp: swap order of lines
lis: insert a line from elsewhere
lrs: replace a line with one from elsewhere

Radamsa

td: delete a node

tr2: duplicate a node

ts1: swap one node with another one

ts2: swap two nodes pairwise

tr: repeat a path of the parse tree

uw: try to make a code point too wide

ui: insert funny unicode

num: try to modify a textual number

xp: try to parse XML and mutate it

ft: jump to a similar position in block

fn: likely clone data between similar positions

fo: fuse previously seen data elsewhere

Radamsa

Mutation patterns (-p)

od: Mutate once

nd: Mutate possibly many times

bu: Make several mutations closeby once

Generators (-g)

stdin: read data from standard input if no paths are given or - is among them

file: read data from given files

random: generate random data

Radamsa Examples

- Use a single XML example installed on your system to generate additional corpus
 - `$ radamsa -o /tmp/out_radamsa/fuzzed_xml_%n.%s -n 100 -r /etc/purple/*.xml -M /tmp/out_radamsa/radamsa.log`
- Pass the -m flag to specify the mutation engine is XML
 - `$ radamsa -o /tmp/out_radamsa_xp/fuzzed_xml_%n.%s -n 100 -r /etc/purple/*.xml -M /tmp/out_radamsa_xp/radamsa.log -m xp`

Radamsa Examples

- Use a single XML example installed on your system to generate additional corpus
 - `$ radamsa -o /tmp/out_radamsa/fuzzed_xml_%n.%s -n 100 -r /etc/purple/*.xml -M /tmp/out_radamsa/radamsa.log`
- Pass the -m flag to specify the mutation engine is XML
 - `$ radamsa -o /tmp/out_radamsa_xp/fuzzed_xml_%n.%s -n 100 -r /etc/purple/*.xml -M /tmp/out_radamsa_xp/radamsa.log -m xp`

Radamsa + coverage = covfu

- The author of radamsa also has a bash script that uses radamsa with AddressSanitizer coverage

```
$ covfu
```

```
Usage: covfu --samples <samples-dir> --work <fuzzing-dir> --timeout <timeout(10)>  
--setsize <setsize(100)> --fuzzer <fuzzer-executable> --target <instrumented-  
target-command> -j <njobs(1)>
```

HINT: more good tools at <https://github.com/aoh?tab=repositories>

Radamsa + Windows?

- I created a fork of radamsa that works on Windows
 - <https://github.com/richinseattle/radamsa/commits/win32-dev>
 - Compiles with mingw/gcc/clang
- New features
 - Can be used from command line as .exe or as shared library
 - Supports in-memory mutation of inputs and outputs

ni

- Ni is a simple radamsa-like mutator written in C
- Possibly easier to hack on or build into a library

```
$ mkdir /tmp/ni_out
$ ./ni -o /tmp/ni_out/passwd_%n -v --count 10 -- /etc/passwd
/tmp/ni_out/passwd_1 = '/etc/passwd': inss arf
/tmp/ni_out/passwd_2 = '/etc/passwd': ajmp jmp arf
/tmp/ni_out/passwd_3 = '/etc/passwd': ajmp
/tmp/ni_out/passwd_4 = '/etc/passwd': jmp arf oversr
/tmp/ni_out/passwd_5 = '/etc/passwd': drop
...
```

Surku

- Surku is a radamsa-like fuzzer written by Atte Kettunen

```
$ nodejs Surku.js
```

```
This is Surku v0.1.1 Help.
```

```
Usage: node Surku.js [arguments] [file ...]
```

```
-o | --output <arg>, file name pattern for outputs, e.g. output/fuzz-%n.foo, required
```

```
-n | --count <arg>, how many files to generate, defaults to 1
```

```
-s | --seed <arg>, which seed to use, defaults to random
```

```
-m | --mutations <arg>, which mutators to use, for defaults use argument "-l" or "--list"
```

```
-l | --list, list default mutators
```

```
-v | --verbose, verbose console-outputs
```

```
-Mm | --maxmutations <arg>, set max mutations per output, default 20
```

```
-mm | --minmutations <arg>, set min mutations per output, default 2
```

Surku

- Surku is a radamsa-like fuzzer written by Atte Kettunen

```
freqString:    {mutatorFunction:freqString,weight:10},
regExpTrick:   {mutatorFunction:regExpTrick,weight:10},
strStuttr:     {mutatorFunction:strStuttr,weight:3},
lineCopy:      {mutatorFunction:lineCopy,weight:2},
lineSwap:      {mutatorFunction:lineSwap,weight:3},
lineMove:      {mutatorFunction:lineMove,weight:3},
lineRepeat:    {mutatorFunction:lineRepeat,weight:3},
wordCopy:      {mutatorFunction:wordCopy,weight:5},
```

Surku

- Surku is a radamsa-like fuzzer written by Atte Kettunen

```
wordRemove:    {mutatorFunction:wordRemove,weight:5},
mutateNumber:  {mutatorFunction:mutateNumber,weight:15},
replaceXMLValuePair: {mutatorFunction:replaceXMLValuePair,weight:20},
strCopyShort:  {mutatorFunction:strCopyShort,weight:2},
strCopyLong:   {mutatorFunction:strCopyLong,weight:1},
strRemove:     {mutatorFunction:strRemove,weight:2},
insertSingleChar: {mutatorFunction:insertSingleChar,weight:2},
insertMultipleChar: {mutatorFunction:insertMultipleChar,weight:2},
```

Surku

- Surku is a radamsa-like fuzzer written by Atte Kettunen

```
replaceSingleChar:    {mutatorFunction:replaceSingleChar,weight:5},
replaceMultipleChar:  {mutatorFunction:replaceMultipleChar,weight:3},
repeatChar:           {mutatorFunction:repeatChar,weight:4},
repeatChars:          {mutatorFunction:repeatChars,weight:3},
bitFlip:              {mutatorFunction:bitFlip,weight:5},
pdfObjectMangle:      {mutatorFunction:pdfObjectMangle,weight:4},
xmlMutator:           {mutatorFunction:require('./xmlMutator.js'),weight:20}
```

BooFuzz

- A modern clone of Sulley fuzzer
 - Designed for attacking network protocols
 - Uses generative approach
-
- This does not have a feedback loop, use it for generating corpus
 - We will explore BooFuzz a bit more in the grammar fuzzing segment

Mutiny Fuzzer and Decept Proxy

- Mutiny and the Decept Proxy are tools for doing man-in-the-middle fuzzing of network protocols.
- These tools allow for very quickly fuzzing specific parts of a recorded packet capture.
- The Mutiny fuzzer wraps radamsa and takes configuration for which fields to fuzz.
- Decept Proxy observes traffic and generates a config for Mutiny

DEMO

- Radamsa fuzzing modes
- Simple mutation fuzzing vs expat_xml_parse
- Mutiny & Decept Proxy

EXERCISE

- Use Mutiny & Decept Proxy to fuzz the vhttpd-challenge webserver

```
vulndev@vulndev-x64 /vulndev/targets $ ./vhttpd
```



OCI SECURITY
INTERVIEW CHALLENGE 2019

Objectives:

- Find the request that allows you to retrieve the source code for this binary (The embedded archive is encrypted with the secret path)
- Identify at least four vulnerabilities in the source code
- Identify one additional vulnerability that is only in the binary
- Write trigger inputs for all discovered vulnerabilities
- Exploit the memory corruption that exists only in the binary
- Optionally recompile with mitigations that show your ability

Coverage Guided Fuzzing

Coverage Guided Fuzzing

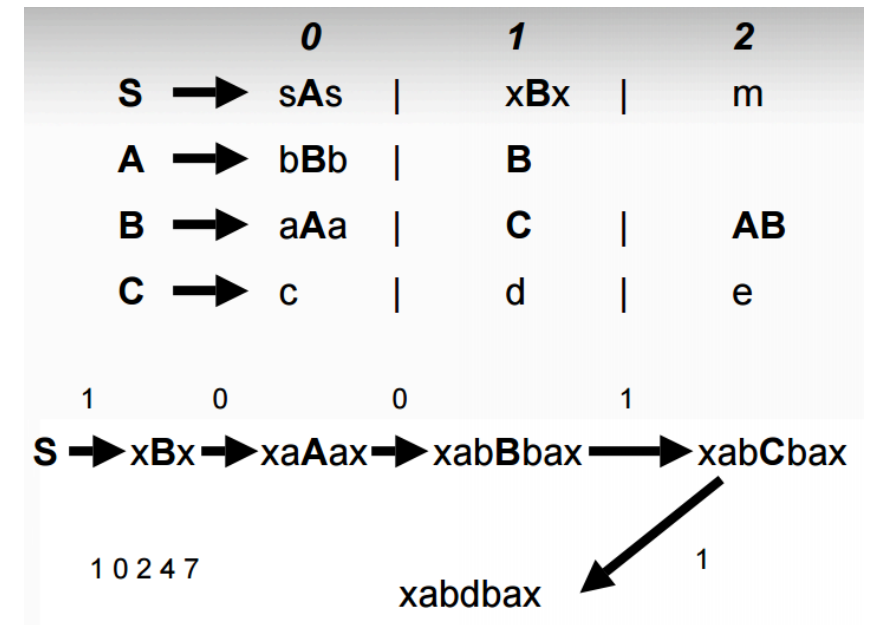
- Creating models of data for fuzzing can be tedious and does not lend itself to data formats that lack an open specification
- A critical turning point for fuzzing was the introduction of a feedback loop between mutation fuzzing and code coverage
- Unique code coverage achieved through random mutation indicates a good mutation that can be kept and continually refined
- Apply evolutionary algorithm concepts of keeping a pool of inputs and assessing fitness of new input compared to existing pool

Coverage Guided Fuzzing

- Incrementally better mutational dumb fuzzing
- Trace while fuzzing and provide feedback signal
- Evolutionary algorithms
 - Assess fitness of current input
 - Manage a pool of possible inputs

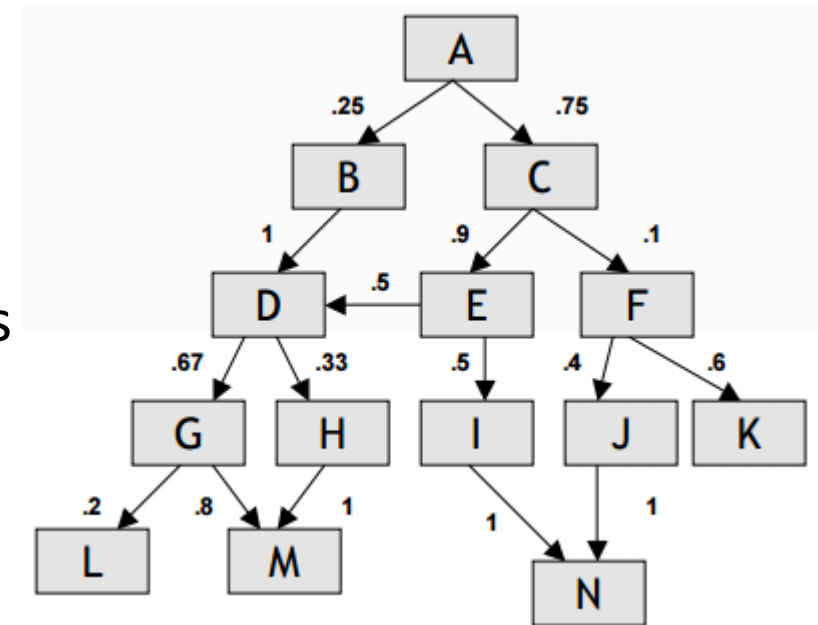
Sidewinder

- Embleton, Sparks, Cunningham 2006
- Features
 - Simple genetic algorithm approach
 - crossover, mutation, fitness
 - Mutated context free grammar
 - instead of sample fuzzing
 - Markov process for fitness
 - Analyzes probability of path taken by sample
 - Block coverage via debugger API
 - Reduced overhead by focusing on subgraphs



Sidewinder

- Embleton, Sparks, Cunningham 2006
- Contributions
 - Genetic algorithms for fuzzing
 - Markov process for fitness
 - System allows selection of target code locations
- Observations
 - Never opensourced
 - Interesting concepts not duplicated



Evolutionary Fuzzing System

- Jared DeMott 2007
- Features
 - Block coverage via Process Stalker
 - Windows Debug API
 - Intel BTF
 - Stored trace results in SQL database
 - Lots of variables required structured storage
 - Traditional genetic programming techniques
 - Code coverage + diversity for fitness
 - Sessions
 - Pools
 - Crossover
 - Mutation

Evolutionary Fuzzing System

- Jared DeMott 2007
- Contributions
 - First opensource implementation of guided fuzzing
 - Evaluated function vs block tracing
 - For large programs found function tracing was equally effective
 - Likely an artifact of doing text based protocols
- Observations
 - Academic
 - Approach was too closely tied to traditional genetic algorithms
 - Not enough attention to performance or real world targets
 - Only targeted text protocols

Guided Fuzzing

- Fast tracing engine
 - Block based granularity
- Fast logging
 - Memory resident coverage map
- Fast evolutionary algorithm
 - Minimum of global population map, pool diversity

honggfuzz

- Robert Swiecki 2010
 - Guided fuzzing added in 2015
- Features
 - Block coverage
 - Hardware performance counters
 - ASanCoverage
 - Bloom filter for trace recording
 - User-supplied mutation functions
 - Linux, FreeBSD, OSX, Cygwin support

```
/bin/bash
-----[ 0 days 00 hrs 14 mins 00 secs ]-----/ honggfuzz 1.3 /-
Iterations : 398,052 [398.05k]
Mode : Feedback Driven Mode (2/2)
Target : './httpd/httpd -X -f /home/jagger/fuzz/apache/dist/conf/h ...'
Threads : 8, CPUs: 8, CPU%: 261% (32%/CPU)
Speed : 323/sec (avg: 473)
Crashes : 90 (unique: 1, blacklist: 0, verified: 0)
Timeouts : [5 sec] 32
Corpus Size : entries: 1,147, max size: 1,048,792, input dir: 8522 files
Cov Update : 0 days 00 hrs 00 mins 05 secs ago
Coverage : edge: 17,019 pc: 410 cmp: 187,266
----- [ LOGS ] -----

Crash (dup): './SIGABRT.PC.7ffff5ef10bb.STACK.18819c8652.CODE.-6.ADDR.(nil).INST
R.mov____0x108(%rsp),%rcx.fuzz' already exists, skipping
[2018-01-18T22:21:22+0100][W][3343] arch_checkWait():308 Persistent mode: PID 21
623 exited with status: SIGNALED, signal: 6 (Aborted)
Persistent mode: Launched new persistent PID: 24520
Crash (dup): './SIGABRT.PC.7ffff5ef10bb.STACK.18819c8652.CODE.-6.ADDR.(nil).INST
R.mov____0x108(%rsp),%rcx.fuzz' already exists, skipping
[2018-01-18T22:21:23+0100][W][3346] arch_checkWait():308 Persistent mode: PID 18
231 exited with status: SIGNALED, signal: 6 (Aborted)
Persistent mode: Launched new persistent PID: 25094
Size:296441 (i,b,hw,edge,ip,cmp): 0/0/0/0/0/1, Tot:0/0/0/17019/410/187266
```

honggfuzz

- Robert Swiecki 2010
 - Guided fuzzing added in 2015
- Contributions
 - First guided fuzzer to focus on hardware tracing support
 - Includes interface for LibFuzzer targets
- Observations
 - Naive seed selection for most algorithms, only the elite survive (OTTES)
 - Some modes use bloom filter
 - Easy to extend, active development

American Fuzzy Lop

- Michal Zalewski 2013
 - Bunny The Fuzzer 2007
- Features
 - Block coverage via compile time instrumentation
 - Simplified approach to genetic algorithm
 - Edge transitions are encoded as tuple and tracked in global map
 - Includes coverage and frequency
 - Uses variety of traditional mutation fuzzing strategies
 - Dictionaries of tokens/constants
 - First practical high performance guided fuzzer
 - Helper tools for minimizing test cases and corpus
 - Attempts to be idiot proof

American Fuzzy Lop

- Michal Zalewski 2013
 - Bunny The Fuzzer 2007
- Features
 - Deterministic bitflip
 - 1, 2, 4, 8, 16, 32 bits
 - Deterministic addition/subtraction
 - Values { 1 – 35 } for each byte, short, word, dword
 - Little endian and big endian
 - Deterministic 'interesting' constant values
 - 27 boundary values
 - Dictionary keywords
 - Havoc
 - Random bitflips, arithmetic, block move/copy, truncate
 - Splice
 - Merge two previously generated inputs

American Fuzzy Lop

- Michal Zalewski 2013
 - Bunny The Fuzzer 2007
- Contributions
 - Tracks edge transitions
 - Not just block entry
 - Global coverage map
 - Generation tracking
 - Fork server
 - Reduce fuzz target initialization
 - Persistent mode fuzzing
 - Builds corpus of unique inputs reusable in other workflows

```
american fuzzy lop 2.52b (dump-prog)

- process timing -
  run time : 0 days, 0 hrs, 2 min, 7 sec
  last new path : 0 days, 0 hrs, 0 min, 48 sec
  last uniq crash : none seen yet
  last uniq hang : none seen yet
- cycle progress -
  now processing : 15 (2.37%)
  paths timed out : 0 (0.00%)
- stage progress -
  now trying : bitflip 2/1
  stage execs : 2054/3119 (65.85%)
  total execs : 155k
  exec speed : 1331/sec
- fuzzing strategy yields -
  bit flips : 0/22.5k, 0/19.3k, 0/19.3k
  byte flips : 0/2417, 0/395, 0/431
  arithmetics : 2/21.1k, 0/21.8k, 0/21.5k
  known ints : 0/1081, 0/4491, 0/8749
  dictionary : 0/0, 0/0, 0/428
  havoc : 1/3828, 0/0
  trim : 0.00%/1287, 83.88%

- overall results -
  cycles done : 0
  total paths : 634
  uniq crashes : 0
  uniq hangs : 0
- map coverage -
  map density : 0.39% / 3.33%
  count coverage : 4.55 bits/tuple
- findings in depth -
  favored paths : 227 (35.80%)
  new edges on : 256 (40.38%)
  total crashes : 0 (0 unique)
  total tmouts : 1 (1 unique)
- path geometry -
  levels : 2
  pending : 627
  pend fav : 221
  own finds : 3
  imported : n/a
  stability : 100.00%

[cpu000: 37%]
```

American Fuzzy Lop

- Michal Zalewski 2013
 - Bunny The Fuzzer 2007
- Observations
 - KISS works when applied to guided fuzzing
 - Performance top level priority in design
 - Source instrumentation can't be beat
 - Evolutionary system hard to beat without greatly increasing complexity / cost
 - Simple to use, finds tons of bugs

American Fuzzy Lop

- Michal Zalewski 2013
 - Bunny The Fuzzer 2007
- Observations
 - Fostered a user community
 - Developer contributions somewhat difficult
 - Current state of the art due to good engineering and feature set
 - Only mutational fuzzer system to have many third-party contributions
 - Binary support via QEMU and Dyninst
 - More robust compiler instrumentations, ASAN support
 - Parallelization, client/server targeting

American Fuzzy Lop

- Fostered a user community
 - afl-qemu
 - afl-dynamoriot
 - afl-pin
 - afl-dyninst
 - afl-utils
 - afl-cov
 - Python AFL
 - TriforceAFL
 - Go-fuzz
 - WinAFL
 - Dozens of academic papers
- We will look at several of these in detail through the course of the training

DEMO

- Compile expat library and expat_xml_parse.c with AFL support
 - `$ CC=afl-gcc CXX=afl-g++ ./configure && make`
 - `$ export LD_LIBRARY_PATH=/path/to/dir/containing/libexpat.so`
 - `$ afl-gcc -o expat_xml_parse expat_xml_parse.c -lexpat`
- Run AFL fuzzer against the newly compiled target using input generated by the radamsa exercise
 - `$ afl-fuzz -i /path/to/inputs -o /path/for/output -- ./expat_xml_parse @@`

WinAFL

- Ivan Fratric July 2016
 - First performant windows evolutionary fuzzer
- Features
 - Its American Fuzzy Lop! For Windows!
 - Windows API port for memory and process creation
 - DynamoRIO based code coverage
 - Filter based on module
 - Block and Edge tracing modes
 - Block tracing by default due to issues with multi-threading
 - Persistent execution mode

WinAFL

- Ivan Fratric July 2016
 - First performant windows evolutionary fuzzer
- Persistence
 - Multiple inputs can be parsed without exiting the process
 - DynamoRIO allows hooking of target function
 - User specifies address and number of arguments
 - On function exit, WinAFL repopulates args and loops function
 - User specifies number of loops before process restart

WinAFL

- Ivan Fratric July 2016
 - First performant windows evolutionary fuzzer

- Persistence is key
 - Restart process each time (disable persistence) ~2.3 exec/s
 - Persist 100 iterations before restart ~72 exec/s
 - Persist 1000 iterations ~123 exec/s
 - Persist 10000 iterations ~133 exec/s

```
WinAFL 1.09 based on AFL 2.43b (test_gdiplus.exe)

+- process timing -----+
|   run time : 0 days, 0 hrs, 0 min, 34 sec   |
|   last new path : 0 days, 0 hrs, 0 min, 5 sec |
| last uniq crash : none seen yet              |
| last uniq hang : none seen yet              |
+- cycle progress -----+
| now processing : 0 (0.00%)                   |
| paths timed out : 0 (0.00%)                 |
+- stage progress -----+
| now trying : bitflip 1\1                     |
| stage execs : 592/5040 (11.75%)              |
| total execs : 1055                           |
| exec speed : 46.05/sec (slow!)               |
+- fuzzing strategy yields -----+
| bit flips : 0/0, 0/0, 0/0                   |
| byte flips : 0/0, 0/0, 0/0                 |
| arithmetics : 0/0, 0/0, 0/0                |
| known ints : 0/0, 0/0, 0/0                 |
| dictionary : 0/0, 0/0, 0/0                 |
|   havoc : 0/0, 0/0                         |
|   trim : 0.00%/302, n/a                    |
+- overall results -----+
|   cycles done : 0                           |
|   total paths : 16                          |
|   uniq crashes : 0                          |
|   uniq hangs : 0                            |
+- map coverage -----+
|   map density : 3.67% / 4.64%               |
|   count coverage : 1.12 bits/tuple          |
+- findings in depth -----+
| favored paths : 1 (6.25%)                   |
| new edges on : 16 (100.00%)                 |
| total crashes : 0 (0 unique)                |
| total tmouts : 0 (0 unique)                |
+- path geometry -----+
|   levels : 2                               |
|   pending : 16                             |
|   pend fav : 1                             |
|   own finds : 15                           |
|   imported : n/a                           |
|   stability : 78.54%                       |
+-----+
```

WinAFL

- WinAFL is built on top of DynamoRIO and requires some manual analysis of your target we will look at its internals and use in the Windows Binary Fuzzing segment

LibFuzzer

- LLVM includes built-in coverage guided fuzzing capability to aid developers
- A user of LibFuzzer writes a single function test harness for the code they wish to exercise and a runtime library takes care of the rest
- We will use LibFuzzer later for fuzzing file and network parsers

Exercise

- Compile `/vulndev/targets/nasm` with AFL support
- Use `/vulndev/corpus/corkami-pocs/PE/bin` as input
- Fuzz `ndisasm`
- Notice any warnings generated by AFL regarding input size
 - We will optimize this later

Exercise

- Pick one of the other simple file parsing targets from `/vulndev/targets`
- Compile with AFL support and begin fuzzing with inputs from `/vulndev/corpus`
- Observe your performance and how many paths are being generated

Advanced File and Network Fuzzing

Advanced Fuzzing

- Today and tomorrow we will be targeting Linux binaries, network clients or servers and Windows binaries with InMemory Fuzzing and Binary Translation
- To modify our targets to work with coverage guided fuzzing we will be using hooks, emulators, or custom harnesses

InMemory Fuzzing

- InMemory Fuzzing executes multiple iterations of fuzzing without exiting a process
- We must somehow wrap a target API(s) with our own logic so that we may re-enter program execution at a desired location
- First we will do this with AFL and LLVM against source code targets
- We will add a harness that can call the code logic we want to exercise

InMemory Fuzzing

- InMemory Fuzzing is also the best way to fuzz complex binary targets
- For binary targets some analysis should be performed to locate functions that take a file path as a parameter
- With a little extra reversing, we can also look for functions that parse sub-structures to improve our performance

InMemory Fuzzing

- WinAFL works by using DynamoRIO to hook functions and can wrap a target function address and fuzz a selected parameter
- We will look at WinAFL in detail after learning more about DynamoRIO internals
- For now, lets look at InMemory Fuzzing on Linux source targets

AFL Persistent Mode

- To greatly increase the speed of fuzzing targets with source code we can use AFL's built in "Persistent Mode"
- Write a standalone harness for a library or modify application code directly

AFL Persistent Mode

harness.c:

```
int main(int argc, char **argv)
{
    ..initialize variables..
    while(__AFL_LOOP(5000)) {
        .. call APIs of interest ..
    }
    return 0;
}
```

```
afl-clang-fast harness.c -o harness -l<library>
```

```
afl-fuzz -i <in> -o <out> -- ./harness @@
```

AFL Persistent Mode

Compile:

```
afl-clang-fast harness.c -o harness -l<library>
```

Run:

```
afl-fuzz -i <in> -o <out> -- ./harness @@
```

AFL Persistent Mode DEMO

- `expat_xml_parse_persist.c`

```
int main(int argc, char *argv[])
{
    if(argc < 2) { printf("usage: %s <xml file>\n", argv[0]); return 1; }
    while(__AFL_LOOP(1000))
    {
        parser = XML_ParserCreate(NULL);
        memset(buf, 0, sizeof(buf));
        parse_xml(argv[1]);
        XML_ParserFree(parser);
    }
    return 0;
}
```

AFL Persistent Mode DEMO

- run-parser-persist.c
- \$ diff -c1 run-parser.c run-parser-persist.c
- *** run-parser.c 2014-01-12 18:24:55.000000000 -0600
- --- run-parser-persist.c 2018-03-18 08:03:54.471890341 -0500
- *****
- *** 32,33 ****
- --- 32,35 ----
-
- + while(__AFL_LOOP(1000))
- + {
- + file = fopen(argv[number], "rb");
- *****

AFL Persistent Mode DEMO

- `run-parser-persist.c`
- `$ diff -c1 run-parser.c run-parser-persist.c`
- `---`
- `*** 56,58 ****`
- `assert(!fclose(file));`
- `!`
- `printf("%s (%d events)\n", (error ? "FAILURE" : "SUCCESS"), count);`
- `--- 58,60 ----`
- `assert(!fclose(file));`
- `! }`
- `printf("%s (%d events)\n", (error ? "FAILURE" : "SUCCESS"), count);`

AFL Persistent Mode Exercise

- Modify your chosen target to include the `__AFL_LOOP`
- You can write a standalone function with the loop and inject a call to it where appropriate in the main application code

LibFuzzer

- LibFuzzer works by using a combination of AddressSanitizer code coverage and a fuzzing harness built into LLVM
- Simply enable the proper compile time arguments and export a magic function prototype..

LibFuzzer

```
bool FuzzMe(const uint8_t *Data, size_t DataSize) {  
    return DataSize >= 3 &&  
        Data[0] == 'F' &&  
        Data[1] == 'U' &&  
        Data[2] == 'Z' &&  
        Data[3] == 'Z';    // :-<  
}  
  
extern "C" int LLVMFuzzerTestOneInput(const uint8_t *Data, size_t  
Size) {  
    FuzzMe(Data, Size);  
    return 0;  
}
```

LibFuzzer DEMO

- Follow along with the libyaml demo in the notes
- Notice how we can now pass a buffer instead of a file
 - This allows us to hook deeper

LibFuzzer DEMO

```
root@addea8de22ec:/vulndev/targets/libyaml/yaml-0.1.5-libfuzzer# ./yamlfuzz /vulndev/corpus/yaml 2>&1 | head
```

```
INFO: Seed: 1353073291
```

```
INFO: Loaded 1 modules (3140 guards): 3140 [0x7de790, 0x7e18a0),
```

```
INFO: 1179 files found in /vulndev/corpus/yaml
```

```
INFO: -max_len is not provided; libFuzzer will not generate inputs larger than 4096 bytes
```

```
INFO: seed corpus: files: 1179 min: 1b max: 4096b total: 402612b rss: 41Mb
```

```
#1180      INITED cov: 1364 ft: 6412 corp: 885/273Kb exec/s: 0 rss: 144Mb
```

```
#1343      REDUCE cov: 1364 ft: 6412 corp: 885/273Kb exec/s: 0 rss: 150Mb L: 1092/4096 MS: 3 ShuffleBytes-ChangeASCIIInt-EraseBytes-
```

```
#1498      REDUCE cov: 1364 ft: 6412 corp: 885/273Kb exec/s: 0 rss: 155Mb L: 104/4096 MS: 3 InsertRepeatedBytes-ShuffleBytes-EraseBytes-
```

```
...
```

```
#103153    REDUCE cov: 1216 ft: 5170 corp: 980/181Kb exec/s: 11461 rss: 189Mb L: 13/3856 MS: 1 EraseBytes-
```

```
#103173    REDUCE cov: 1216 ft: 5170 corp: 980/181Kb exec/s: 11463 rss: 189Mb L: 31/3856 MS: 1 EraseBytes-
```

LibFuzzer Exercise

- Now modify the AFL_LOOP exercise to include a `LLVMFuzzerTestOneInput()` function
- Notice the performance differences

LibFuzzer Exercise

- `docker run --cap-add SYS_PTRACE -ti libfuzzertutorial/prebuilt`
- `clang++ -g -fsanitize=address -fsanitize-coverage=trace-pc-guard FTS/tutorial/fuzz_me.cc libFuzzer.a && ./a.out 2>&1 | grep ERROR`
- Follow the official libFuzzer tutorial on Google's GitHub
 - <https://github.com/google/fuzzer-test-suite/blob/master/tutorial/libFuzzerTutorial.md>

AFL vs ELF Binaries

- By default AFL does not target binaries
- We now have multiple options on Linux using QEMU or DYNINST

AFL QEMU

- AFL's QEMU support adds the ability to trace unmodified binaries
- At its core QEMU is a software CPU emulator, the CPU emulator itself is called the Tiny Code Generator (TCG)
- Modifications to the TCG have been made to log branches in an AFL compatible byte map

AFL QEMU

- By default only the first binary loaded (application binary) will be targeted
- Lcamtuf recommends compiling support libraries you want to fuzz statically into the application, however you want to link system libraries dynamically
- Alternatively use `AFL_INST_LIBS=1` to instrument all code

AFL QEMU

- Use the -Q command line switch to make AFL use QEMU mode
- We may need to increase the amount of memory a fuzzed process is allowed to use with the -m <size> switch e.g. -m 1G
- `afl-fuzz -Q -i <in> -o <out> -- /normal/binary @@`

AFL QEMU Exercise

- Try fuzzing a non-instrumented distribution package with AFL such as tiffinfo
- Try fuzzing some closed source commercial binaries such as the Oracle Outside In Technology parsers in /vulndev/target/oit-ix
- Hint: Since OIT are shared libs you will need to pass the AFL_INST_LIBS=1

AFL-DYNINST

- DynInst is a powerful static binary rewriting framework
- Talos VulnDev team has implemented a plugin to modify binaries on disk to include block based code coverage hooks and AFL compatible logging
- We will examine the internals in detail tomorrow after the DBI section

AFL-DYNINST Exercise

- `./afl-dyninst.sh -i `which tiffinfo` -o tiffinfo.dyninst -l libtiff.so.5 -f`
- Inserting init callback.
- Skipping library: tiffinfo
- Skipping library: tiffinfo.c
- Skipping library: getpid.c
- ...
- Instrumenting module: libtiff.so.5

AFL-DYNINST Exercise

- Skipping library: libAflDyninst.cpp
 - Skipping library: libAflDyninst.so
 - Saving the instrumented binary to tiffinfo.dyninst ...
 - Did a total of 12859 basic block insertions
 - All done! Happy fuzzing!
-
- `$ ls *tiff*`
 - `libtiff.so.5 tiffinfo.dyninst`
-
- `./afl-fuzz-dyninst.sh -mnone -i /vulndev/afl/testcases/images/tiff/ -o /tmp/afl-tiff -- ./tiffinfo.dyninst @@`

AFL-DYNINST Exercise

- Make a new working copy of the OIT XI libraries
- Instrument with afl-dyninst
- Notice the performance differences between QEMU and DYNINST

Improving File Fuzzing

Input Selection

- Input is a numerical set
- Input parsers are (should be) state machines
 - Specifications described using FSM
 - Actual parser code typically not implemented using FSM
 - LangSec Paper on high performance FSM Parsers
 - <http://www.cs.dartmouth.edu/~pete/pubs/LangSec-2014-fsm-parsers.pdf>
- Goal: search space and discover new transitions
- Each search is computationally expensive
 - We need to optimize for time

Input Selection

- Optimize input selection
 - File size is very important
 - Mutations are more meaningful with smaller input size
 - Smaller inputs are read and parsed quicker
 - Some test generation approaches utilize large amounts of memory per-input-byte
 - Specific feature set per input allows for focused targeting
 - Handcrafted or minimized samples
 - Feedback fuzzing or concolic testing automates creation of unique small inputs with different features, we will experiment with this the last day

AFL-TMIN

- American Fuzzy Lop ships a tool to do testcase minimization for you

```
/vulndev/corpus/tob-pe-parse-output $ \  
afl-tmin -i /ramdisk/combined_inputs/standard.exe -o min.exe -- tob-pe-parse-  
output/dump-prog @@  
afl-tmin 2.52b by <lcamtuf@google.com>
```

```
[+] Read 10880 bytes from '/ramdisk/combined_inputs/standard.exe'.  
[*] Performing dry run (mem limit = 50 MB, timeout = 1000 ms)...  
[+] Program terminates normally, minimizing in instrumented mode.  
[*] Stage #0: One-time block normalization...  
[+] Block normalization complete, 10240 bytes replaced.
```

AFL-TMIN

[*] Stage #1: Removing blocks of data...

Block length = 1024, remaining size = 10880

Block length = 512, remaining size = 8832

Block length = 256, remaining size = 8704

Block length = 128, remaining size = 8704

Block length = 64, remaining size = 8704

Block length = 32, remaining size = 8704

Block length = 16, remaining size = 8704

Block length = 8, remaining size = 8704

Block length = 4, remaining size = 8704

Block length = 2, remaining size = 8704

Block length = 1, remaining size = 8704

[+] Block removal complete, 2176 bytes deleted.

AFL-TMIN

[*] Stage #2: Minimizing symbols (88 code points)...

[+] Symbol minimization finished, 73 symbols (164 bytes) replaced.

AFL-TMIN

```
[*] Stage #3: Character minimization...  
[+] Character minimization done, 432 bytes replaced.  
[*] --- Pass #2 ---  
[*] Stage #1: Removing blocks of data...  
    Block length = 1024, remaining size = 8704  
    Block length = 512, remaining size = 8704  
    Block length = 256, remaining size = 8704  
    Block length = 128, remaining size = 8704  
    Block length = 64, remaining size = 8704  
    Block length = 32, remaining size = 8704  
    Block length = 16, remaining size = 8704  
    Block length = 8, remaining size = 8704  
    Block length = 4, remaining size = 8704  
    Block length = 2, remaining size = 8704  
    Block length = 1, remaining size = 8704  
[+] Block removal complete, 0 bytes deleted.
```

AFL-TMIN

File size reduced by : 20.00% (to 8704 bytes)

Characters simplified : 124.49%

Number of execs done : 1532

Fruitless execs : path=943 crash=0 hang=0

[*] Writing output to 'min.exe'...

[+] We're done here. Have a nice day!

```
/vulndev/corpus $ ll standard.exe min.exe
```

```
-rw----- 1 vulndev vulndev 8704 Dec 6 02:43 min.exe
```

```
-rw-r--r-- 1 vulndev vulndev 10880 Dec 6 02:29 standard.exe
```

Input Selection

- Corpus Minimization uses code coverage traces to eliminate inputs that do not exercise new areas of code
 - If two inputs achieve similar code coverage, keep the smaller of the two
 - Use a heuristic to determine required new coverage requirements per file
- Various strategies were studied by CMU
 - Optimizing Seed Selection for Fuzzing - USENIX 2014
 - <https://www.usenix.org/system/files/conference/usenixsecurity14/sec14-paper-rebert.pdf>

AFL-CMIN

- American Fuzzy Lop ships a tool to do corpus minimization for you

```
/vulndev/corpus $ mkdir /ramdisk/combined_inputs
/vulndev/corpus $ cp tob-pe-parse-output/out/queue/* /ramdisk/combined_inputs/
/vulndev/corpus $ cp clamav_corpus/exe_seed/* /ramdisk/combined_inputs/
/vulndev/corpus $ cp corkami-pocs/PE/bin/* /ramdisk/combined_inputs/
/vulndev/corpus $ ls -1 /ramdisk/combined_inputs | wc -l
2318
/vulndev/corpus/tob-pe-parse-output $ \
afl-cmin -i /ramdisk/combined_inputs/ -o /ramdisk/pe_minset -- ./dump-prog @@
corpus minimization tool for afl-fuzz by <lcamtuf@google.com>
```

AFL-CMIN

[*] Testing the target binary...

[+] OK, 380 tuples recorded.

[*] Obtaining traces for input files in '/ramdisk/combined_inputs/'...

Processing file 2318/2318...

[*] Sorting trace sets (this may take a while)...

[+] Found 9920 unique tuples across 2318 files.

[*] Finding best candidates for each tuple...

Processing file 2318/2318...

[*] Sorting candidate list (be patient)...

[*] Processing candidates and writing output files...

Processing tuple 9920/9920...

[+] Narrowed down to 631 files, saved in '/ramdisk/pe_minset'.

Fuzzing Dictionaries

- Good fuzzers support injecting static user supplied tokens as well as random data
 - AFL, LibFuzzer and DrFuzz support using dictionaries
 - Classic fuzzers used dictionaries of known bad integers and strings
- AFL includes libtokencap, an LD_PRELOAD library that hooks memory comparison APIs and outputs the value the program is expecting
 - strcmp
 - strncmp
 - strcasecmp
 - strncasecmp
 - memcmp
 - strstr
 - strcasestr

Fuzzing Dictionaries

From the manual:

To use the library, you **need** to make sure that your fuzzing target is compiled with `-fno-builtin` and is linked dynamically. If you wish to automate the first part without mucking with CFLAGS in Makefiles, you can set `AFL_NO_BUILTIN=1` when using `afl-gcc`. This setting specifically adds the following flags:

- `-fno-builtin-strcmp` `-fno-builtin-strncmp` `-fno-builtin-strcasecmp`
- `-fno-builtin-strcasencmp` `-fno-builtin-memcmp` `-fno-builtin-strstr`
- `-fno-builtin-strcasestr`

Fuzzing Dictionaries

- Relying on hooking APIs will not always work
- Libtokencap requires compiling from source to be useful for most targets
- A better option:
 - Source code: use an llvm plugin
 - <https://doar-e.github.io/blog/2016/11/27/clang-and-passes/>
 - Binary: use a DBI to do instruction level analysis
 - We can play with this later

Fuzzing Dictionaries

```
$ head -50 /vulndev/afl/dictionaries/jpeg.dict
# AFL dictionary for JPEG images
# -----
# Created by Michal Zalewski <lcamtuf@google.com>
```

```
header_jfif="JFIF\x00"
header_jfxx="JFXX\x00"
```

```
section_ffc0="\xff\xc0"
section_ffc2="\xff\xc2"
...
```

Use the `-x` flag with `afl-fuzz` to use the dictionaries

Exercise

- Run corpus minimization on the inputs you have chosen to use for your target library
- `$ afl-cmin -i /path/to/original/inputs -o /path/to/save/minimized/corpus -- <target cmdline>`
- AFL warned that some of the PE binaries from corkami's corpus were too large, select one and run the testcase minimizer on it:
- `$ afl-tmin -i /path/to/large/input -o /path/to/minimized/output -- <target cmdline>`

Exercise

- Since we will continue to improve our fuzzing technique on the library you have chosen, now would be a good time to extract constants and add them to a dictionary
- Compile your target with `AFL_NO_BUILTIN=1`
- `$ AFL_NO_BUILTIN=1 ./configure && make`
- Compile libtokencap
- `$ cd /vulndev/afl/libtokencap && make`

Exercise

- Run your target with existing corpus to extract tokens:

```
$ export AFL_TOKEN_FILE=$PWD/tokencap_output.txt
```

```
$ for i in <out_dir>/queue/id*; do
```

```
    LD_PRELOAD=/vulndev/afl/libtokencap/libtokencap.so \
```

```
    /path/to/target/program [...params, including $i...]
```

```
done
```

```
$ sort -u tokencap_output.txt > <filetype>_dictionary.txt
```

- Use the new dictionary in your fuzzing with the
'-x /path/to/dictionary' parameter

Performant Fuzzing

Performant Fuzzing

- We have now learned some tools and approaches for hitting target code
- Lets look further at performance characteristics of fuzzing

Process Creation

- Fuzzing creates lots of processes
- One key to fast fuzzing is to be able to create processes quickly and efficiently
- Linux is king here, since `fork()` is cheap due to using copy-on-write memory mappings to clone a process
- OSX and Windows are a fail on this front

Process Creation

- OSX and Windows are a fail on this front
- The lesson learned here is to use in-memory fuzzing techniques to hook top level functions that receive the file path
- In Linux, it also helps to inject a shared library that will cause the program to fork after the program has loaded libraries and performed any initialization (we will see how to do this generically later)

Process Creation

- Also use taskset on Linux or SetThreadAffinity() on Windows to stop the kernel scheduler from allowing your process to switch CPU cores. This will ensure you don't lose your caches

linux

10000 fork()

0.763s → 13106 exec/sec

10000 fork/exec(/bin/false)

2.761s → 3621 exec/sec

10000 fork/exec(/bin/false) w/ taskset

2.073s → 4823 exec/sec

cygwin

10000 fork()

29.954s → 333 exec/sec

10000 fork/exec(/bin/false)

63.898s → 156 exec/sec

RtlCloneUserProcess (older hardware)

10000 fork()

17.457s → 574 exec/sec

Disk I/O

- Fuzzing reads and writes a lot of files
- First, if you don't have copy-on-write or your program is too large to fit into cache, you will have I/O to worry about just from the program executing
- Second, each iteration is generating a new file and some amount of I/O is performed on that file by the target process
- Disable swap and use ramdisk

Disk I/O

```
echo "    disabling swap disks... (this may take some time)"
sudo bash -c "swapoff -a"

# create ramdisk
sudo mkdir -p /ramdisk
sudo chown `whoami` /ramdisk
echo "    creating 2GB tmpfs at /ramdisk..."
if [ "`mount | grep /ramdisk`1" == "1" ]
then
    sudo bash -c 'mkdir -p /ramdisk'
    sudo mount -t tmpfs -o size=2G tmpfs /ramdisk
else
    echo "        ramdisk already mounted at /ramdisk, skipping...";
fi

# don't forget to copy your files...
```

Kernel Config

- Next we want to disable ASLR, CPU Scaling, and crash handlers

```
echo "      disabling system crash notifiers..."
sudo bash -c 'echo core > /proc/sys/kernel/core_pattern'
echo "      disabling cpu scaling..."
sudo bash <<"EOF"
for i in $(seq `nproc`); do
    let "cpu_idx = $i - 1";
    echo performance > /sys/devices/system/cpu/cpu$cpu_idx/cpufreq/scaling_governor;
done
EOF
echo "      disabling address space randomization..."
sudo bash -c "echo 0 > /proc/sys/kernel/randomize_va_space"
```

Input Generation

- Depending on your inputs, you may or may not want to do a deterministic pass flipping each bit or byte.
- This depends both on the size of your input and the robustness of your target

Tracing Engine

- Tracing engine selection is critical
 - AFL-QEMU
 - Uses QEMU userland block tracing
 - Statically compiled binaries
 - Linux only
 - Readpng: ~860 ex/s vs ~3800 afl-gcc – 4.5x slower
 - AFL-DynInst
 - Static binary rewriting
 - Dynamically compiled binaries
 - Linux only
 - Readpng: ~2400 ex/s vs ~3300 afl-gcc – 1.3x slower

Trace Logging

- Earlier tooling used various attempts to diff lists basic blocks
- One of the main keys to AFL's performance is the use of a simplified bloom filter to track visited edges

```
cur_location = (block_address >> 4) ^ (block_address << 8);  
shared_mem[(cur_location ^ prev_location) % sizeof(shared_mem)]++;  
prev_location = cur_location >> 1;
```

- This technique propagated to visualization and analysis tools like Lighthouse as well

Fuzzing Pipeline

- Different techniques are useful at different phases of your fuzzing cycle
- Your fuzzing pipeline should not use memory checkers during the initial corpus generation phase
- Once you are achieving good code coverage on the areas you are trying to reach, then add in memory hardening to ensure bugs are caught

Fuzzing Pipeline

- Some parsers are slower than others, especially if you are targeting closed source software
- Use an open source parser to generate initial corpus and then begin fuzzing your actual target of interest
- Parsers will implement different parts of the specification, but you will accelerate your efforts this way and sometimes find “free” bugs

Fuzzing Pipeline

- Some more advanced techniques we will cover on the last day such as concolic execution are generally CPU bound and may take a long time per iteration
- The deeper the code paths are, the longer it will take to solve them
- Sometimes it's best to use concolic or symbolic execution earlier in the input generation phase, especially if you do not have adequate starting corpus

Day 1 Wrap-Up Discussion